Week 1 First Gemini API Call

-D. Mohit Venkat Sai

-2520030106

Aim

To connect a Python program to the Gemini API, send a text prompt, and display the model's response.

Objectives

- Store an API key in a .env file instead of writing it in Python code.
- Install and use Google's Python SDK.
- Send a prompt to a Gemini model.
- Identify and troubleshoot common setup and API errors.

Software and Requirements

- Visual Studio Code
- Python and the project virtual environment
- A Gemini API key from Google AI Studio
- Internet access
- Python packages: google-genai and python-dotenv

Procedure

1. Opened the Week 1 project folder in Visual Studio Code.
2. Activated the project's virtual environment.
3. Installed the required Python packages:

```
python -m pip install google-genai python-dotenv
```

4. Created a .env file in the project folder and added the Gemini API key:

```
GEMINI_API_KEY=your_actual_api_key
```

5. Created main.py with the program below.
6. Ran the program from the VS Code terminal:

```
python main.py
```

7. After an initial temporary service error, reran the program. Gemini generated a response, which was printed in the terminal.

Program Code:

```
from dotenv import load_dotenv

from google import genai

import os


# Step 1: Load API key from .env

load_dotenv()


# Step 2: Read Gemini API key

api_key = os.getenv("GEMINI_API_KEY")


if not api_key:

  raise ValueError("Gemini API key not found!")


# Step 3: Initialize Gemini client

client = genai.Client(api_key=api_key)


prompt = input("Enter Question: ")

# Step 4: Make first LLM call

response = client.models.generate_content(

  model="gemini-3.5-flash-lite",

  contents=prompt

)

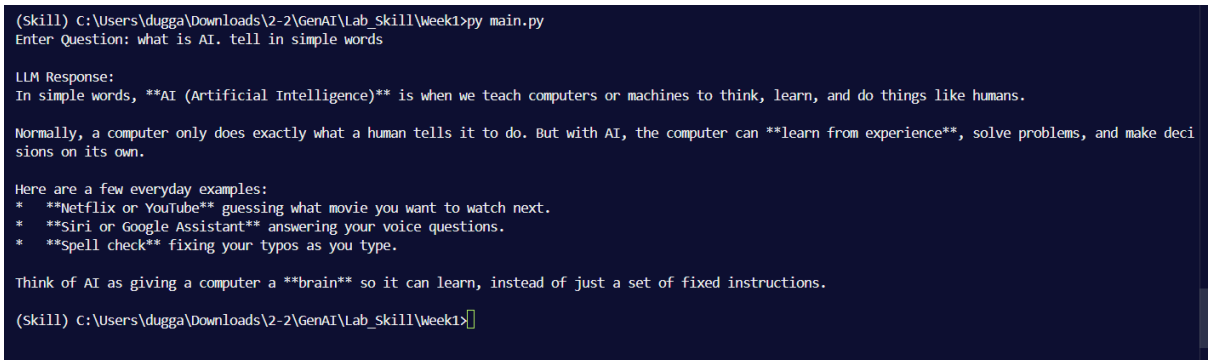

# Step 5: Display response

print("\nLLM Response:")

print(response.text)
```

The API key was stored locally in .env and was not included in the Python source code.

Result and Observation



The program successfully sent a prompt to Gemini and displayed a natural-language response in the terminal. The response explained Artificial Intelligence in simple terms. The exact wording was generated dynamically.

An initial request returned a **503 UNAVAILABLE** high-demand error. After retrying, the request completed successfully.

Troubleshooting

An import error occurred when Python was not using the project's virtual environment. Activating .venv allowed Python to use the installed google-genai and python-dotenv packages.

The 503 response indicated temporary model service demand. Retrying later allowed the request to succeed.

Conclusion

The lab successfully connected a Python program to the Gemini API. It demonstrated how to load an API key securely from .env, submit a prompt with the Google GenAI SDK, and display the generated response. The initial temporary 503 error was resolved by retrying the request.